Certified finite packings of Platonic solids: a generalized soft-to-rigid search and its computational development

Alejandro Zarzuelo Urdiales

11 October 2026

Source snapshot. All project-specific algorithms, records, and historical quantities in this manuscript refer to alejandrozu/platonic-packing, commit `e15b496ae7306287e0e70bb5b14be79f5dfdde84`. The numerical search described here ended on 11 October 2026 at 17:38 CEST, with its final records published at 17:39 CEST. The subsequent documentary commit contains the full project report. This manuscript is a scientific reconstruction of that experiment, rather than a claim that all historical computations have been rerun.

Abstract

We describe an open computational framework for placing congruent regular polyhedra in the smallest homothetic regular-polyhedral container found by numerical search. A cube-packing procedure introduced by Yohei Nakajima is generalized by replacing a convex piece P, with a centered inscribed ball of radius ρ, by $S_r = (1 - r/\rho)P \oplus B(0, r)$. This family interpolates from the inscribed ball to the full piece, remains inside the piece, and grows monotonically as the rounding radius decreases. Search combines overlap-penalized heavy-ball dynamics, analytic rigid-motion gradients, numerical legalization, constrained local tightening, and archive-based perturbations. A separate exact checker proves strict containment and pairwise separation of the stored configurations using arithmetic in $\mathbb{Q}(\sqrt{2}, \sqrt{5})$; numerical optimization is not used as evidence of validity. The experiment covers five same-solid problems and four dual-pair problems for every integer population from 2 to 40, producing 351 certified upper bounds. Approximately 121,109 autonomous attempts followed 1,912 initial raw runs. We document the development sequence, geometric derivations, shaking criterion, solver comparisons, unsuccessful approaches, and reproducibility limits. The principal result is a general search-and-certification method with an inspectable data archive. Neither global optimality nor a physical theory of crystallization follows from the computations.

Keywords: finite packing; regular polyhedra; convex polytopes; homotopy; separating axes; constrained optimization; exact arithmetic; basin hopping.

1 Problem, scope, and intellectual provenance

Let A denote a canonical unit-edge container and B a canonical unit-edge piece, both centered at the origin. Define

$$s^*(A, B, n) = \inf \{ s > 0 : \; c_i + R_i B \subseteq sA, \quad \operatorname{int}(c_i + R_i B) \cap \operatorname{int}(c_j + R_j B) = \varnothing \; (i \neq j), \quad R_i \in SO(3) \} .$$

The optimization variables are the positions and orientations of the pieces and the container edge ratio. The container orientation is fixed without loss of generality because the entire config-

uration can be rotated together. Each Platonic solid is achiral, so allowing reflected copies would not enlarge the set of congruent placements. For a fixed finite population, compactness gives attainment: choose any finite feasible upper bound, bound all centers by its container, use compactness of $SO(3)$, and pass to a convergent subsequence. Containment and absence of interior overlap are closed conditions for these convex pieces.

The experiment considers tetrahedra, cubes, octahedra, dodecahedra, and icosahedra. It searches the five cases $A = B$, cubes in octahedra, octahedra in cubes, dodecahedra in icosahedra, and icosahedra in dodecahedra. Thus nine ordered piece/container problems, each with $2 \le n \le 40$, give 351 cases. The geometry architecture supports all 25 ordered Platonic pairs, but the remaining 16 were not comprehensively searched. Throughout, the piece edge is one; an equal numerical edge ratio in two different mixed problems does not imply equal physical container volume.

Every valid placement gives an upper bound $U(A, B, n) \ge s^*(A, B, n)$. The universal lower bound used in the archive is

$$s^*(A, B, n) \ge \left(\frac{n \operatorname{vol}(B)}{\operatorname{vol}(A)} \right)^{1/3}.$$

For the same-solid problems this reduces to $n^{1/3}$. Deleting any piece proves $s^*(A, B, n) \le s^*(A, B, n+1)$. Neither fact implies that a local optimizer has found the minimum, and repeated numerical convergence does not supply the missing lower bound.

The immediate starting point was Nakajima's open-source square/cube packing project. Its three-dimensional implementation rounds cubes, compresses the rounded objects, gradually removes the rounding, and tightens promising configurations using hard contact constraints. Its released cube record is a strictly separated placement of 12 unit cubes in a cube of side 2.9315185094797. The present work adopts this search architecture and extends its geometric representation and certification to general Platonic pieces and containers. Nakajima's software and credited release are cited explicitly, including the retained MIT attribution. Nakajima repository.

Zarzuelo expanded the scope to the Platonic solids and their dual-pair containers and introduced perturbations of already dense configurations into the experimental program. Basin hopping itself is an established technique. Perturbation followed by local minimization is an established global-search strategy; the contribution here is its particular use with polyhedral rounding, a structural shaking criterion, a monotone finite-packing archive, and exact validation. Wales and Doye, 1997.

General polyhedral fitting also has substantial antecedents. Egeblad, Nielsen, and Brazil studied translational packing of arbitrary polytopes in polytope containers using local and guided local search. Romanova, Bennell, Stoyan, and Pankratov developed nonlinear optimization for concave polyhedra with continuous rotations in a minimum-volume cuboid. Callisaya and Kari formulated congruent-polytope packing in bounded convex domains using isometries and separating hyperplanes, with multistart numerical optimization. These differ in rotation assumptions, container objectives, and experimental coverage, but establish that generic geometry, nonlinear fitting, and separator variables predate this project. The present claim is the documented generalization of Nakajima's specific continuation implementation, coupled to the finite Platonic dataset and exact checking, not the invention of polyhedron packing or the first generic packing optimizer. Egeblad et al., 2009, Romanova et al., 2018, Callisaya and Kari, 2025/2026.

2 Development sequence and evidence base

The experiment unfolded in several stages, all dated in CEST. The full narrative is preserved in the frozen project report; the methods below also follow the executable source, rather than assuming that every prose summary remained current.

On 9 October, between 01:36 and approximately 01:58, the original cube code was studied, exact canonical solids were constructed, a generalized distance kernel was implemented, and the simulator, tightener, and two checkers were validated. A first screen beginning at 01:50 covered five same-solid cases and populations 2–12, using eight seeds per case. Cube regression recovered the familiar edge-2 grid values for populations through eight and the $2 + 1/\sqrt{2}$ construction for nine. The earliest non-cube runs already found two-piece same-solid configurations below edge ratio two.

At 02:30 the requested scope expanded to populations through 20, prioritizing the four non-cube same-solid problems. Geometry profiling, candidate screening, deletion of pieces from larger packings, and constructive lattice seeds followed. The initial batch archive ultimately contains 1,912 raw runs, rather than the approximate 1,770 quoted in some contemporary notes. By the first handoff at 08:14, 76 records covered the four non-cube same-solid cases for populations 2–20, and all passed both checkers.

On 9 October between 11:31 and 12:20, profiling motivated replacing many full restarts with perturbations of already tightened packings. The autonomous engine introduced a best configuration and a small alternative pool per case, mixed perturbation moves, adaptive tightening gates, a scheduler, and an explicit monotonicity invariant across populations. Search began at 12:02; all 351 cases had a candidate by 14:16, and a complete certified record set was published at 16:39 after a publishing failure was repaired.

The next day, at 19:14–20:14, five perturbation settings were compared by randomized interleaving. Small perturbations from the best packing gave the highest observed rate of new archive bests. Public coordinate files were also imported as starting configurations, with lineage preserved. Three imported configurations improved promptly, including one by tightening alone. Additional bounded laptop runs and GitHub Actions workers were added overnight. A baseline taken at 03:23 on 11 October supported a 14-hour analysis; the final search policy was adjusted at 17:20 and the workers stopped at 17:38.

The chronology matters because the method was adaptive. Initial schedules, final policy weights, preliminary numerical values, and the mature archive should not be described as one unchanged experiment. In particular, the old mathematical notes contain first-night values later improved by the autonomous engine. This manuscript distinguishes historical observations from the final frozen configurations.

3 Methodology: computational tools and exact geometry

The numerical implementation uses Node.js for soft-shape dynamics and geometric distance kernels, Python for orchestration and exact checking, NumPy for numerical arrays, and SciPy for constrained local optimization and linear programming. The certificate replay requires only the Python standard library. Claude (Anthropic) was used as a computational development tool for implementing, debugging, and operating the search software under the author's direction. OpenAI Codex was used as a research tool for source inspection, verification replay, structural-analysis review, and preparation of the mathematical exposition. These tools generated candidate code and analysis; feasibility is established by the independently rerunnable exact checks of the stored coordinates. Tool use is reported here as part of the computational procedure.

3.1 Canonical geometry and numerical units

The exact representation in `solids.py` uses the biquadratic field

$$F = \mathbb{Q}(\sqrt{2}, \sqrt{5}) = \{a + b\sqrt{2} + c\sqrt{5} + d\sqrt{10} : a, b, c, d \in \mathbb{Q}\}.$$

Writing $\varphi = (1 + \sqrt{5})/2$, the canonical vertex sets are:

Solid	Unit-edge canonical vertices	V, E, F	Inradius squared
Tetrahedron	$(\sqrt{2}/4)(\pm 1, \pm 1, \pm 1)$, sign product $+1$	4, 6, 4	$1/24$
Cube	$(\pm 1/2, \pm 1/2, \pm 1/2)$	8, 12, 6	$1/4$
Octahedron	$\pm(\sqrt{2}/2)e_j$	6, 12, 8	$1/6$
Dodecahedron	$(\varphi/2)(\pm 1, \pm 1, \pm 1)$, plus cyclic $(0, \pm 1/2, \pm\varphi^2/2)$	20, 30, 12	$5/8 + 11\sqrt{5}/40$
Icosahedron	cyclic $(0, \pm 1/2, \pm\varphi/2)$	12, 30, 20	$7/24 + \sqrt{5}/8$

Here F in the count column means number of faces, whereas the field is denoted by the same conventional letter only in the equations above. Each coordinate sign combination stated in the table is included. The tetrahedron is the only noncentrally symmetric piece in this collection, a fact that affects projected-interval calculations.

Edges are the vertex pairs at the minimum exact squared distance. Supporting face planes are derived from triples of vertices, keeping planes for which every vertex lies on one side. To reduce startup cost, a floating prescreen rejects clearly nonsupporting triples; every retained plane is then checked exactly. Face loops are ordered by their edge incidence. Exact self-checks verify unit edges, the known vertex/edge/face counts, Euler's relation, supporting planes, the face loops, equal face distances from the center, and incidence of every edge with two listed faces. Thus the floating prescreen can cause a failed construction if it omits necessary facets; it cannot silently replace the verified canonical solid with an unchecked numerical mesh.

Search uses a different scale. Each piece is multiplied by $e_B = 1/(2\rho_B)$, so its inscribed sphere has radius $1/2$. The container is represented at the same internal edge scale, with unit normals N_f, face offsets β_f, and $\rho_C = \min_f \beta_f$. Its variable size L means

$$N_f \cdot x \le \frac{L}{2}\,\mathrm{off}_f, \qquad \mathrm{off}_f = \beta_f/\rho_C, \qquad s = L/(2\rho_C).$$

Every Platonic container has $\mathrm{off}_f = 1$, but the representation preserves the offsets explicitly. Dividing internal centers by e_B returns unit-edge record coordinates. Normalization by the in-sphere diameter lets the same dimensionless search constants operate on all five piece shapes. It does not make a fixed translational noise amplitude equal in edge units across shapes. For example, noise 0.004 in internal units is about 0.0016 tetrahedron edges and 0.0089 dodecahedron edges. Structural shaking targets, by contrast, are explicitly divided by e_B.

4 Convex-body continuation from balls to polyhedra

For a compact convex piece P containing $B(0, \rho)$, define

$$S_r = (1 - r/\rho)P \oplus B(0, r), \qquad 0 \le r \le \rho.$$

The symbol $\oplus$ denotes Minkowski addition. At $r = \rho$, the core has shrunk to a point and $S_\rho = B(0, \rho)$; at $r = 0$, $S_0 = P$. This formula reduces to a cube of half-side $1/2 - r$, expanded by a radius-r ball, when P is a unit cube.

Proposition 1. Each S_r is contained in P, and $S_r \subseteq S_{r'}$ whenever $r \geq r'$.

Proof. Set $t = r/\rho$. A point of S_r has the form $(1-t)p + tb$, with $p \in P$ and $b \in B(0, \rho) \subseteq P$; convexity proves containment. For a unit vector u, the support function is

$$h_{S_r}(u) = (1 - r/\rho)h_P(u) + r = h_P(u) - (r/\rho)(h_P(u) - \rho).$$

Since $h_P(u) \geq \rho$, this support value cannot decrease when r decreases. The order of support functions is the inclusion order for compact convex bodies. This proves nesting. □

The family supplies a geometric relaxation: a feasible placement of the full pieces is feasible for every softer member at the same poses. Consequently, the globally minimal container for the softened problem is no larger than that for the rigid problem. The nesting theorem does not assert that the numerical optimizer follows global minima, that a particular jammed soft configuration can be sharpened without rearrangement, or that the container size decreases at every dynamics step. The algorithm changes centers, rotations, and container size while allowing penalized violations. Legalization is required at its end.

The pair penalty can nevertheless be derived exactly at the geometric level. For two compact convex cores K, L, let $\mathrm{sd}(K, L)$ be their Euclidean distance when disjoint, zero at contact, and minus their minimum translational penetration depth when overlapping. Set $C = K - L$. Then

$$\mathrm{sd}(K, L) = -\min_{\|u\|=1} h_C(u).$$

If the origin is outside C, the minimizing support direction is opposite the nearest-point direction, giving minus the distance from the origin to C. If the origin is inside C, the minimum support value is the radius of the largest origin-centered ball in C, equivalently the shortest distance to its boundary. This is the translation required to reach a separating placement. The same formula extends continuously to contact and to the point cores at the spherical endpoint.

Proposition 2. With the preceding signed convention,

$$\mathrm{sd}(K \oplus B(0, r), L \oplus B(0, r)) = \mathrm{sd}(K, L) - 2r.$$

Proof. The Minkowski difference becomes $C \oplus B(0, 2r)$, whose support function is $h_C(u) + 2r$ on the unit sphere. Substitute in the signed-support formula. □

Therefore $(2r - \mathrm{sd}(K, L))_+^2$ is the squared penetration depth of the rounded shapes. Here $x_+ = \max(x, 0)$. For unsigned distance, the correct identity is $\mathrm{dist}(K \oplus B_r, L \oplus B_r) = \max(0, \mathrm{dist}(K, L) - 2r)$ when the cores are disjoint; omitting the maximum would incorrectly permit negative unsigned distances. In internal units, $\rho = 1/2$ and the core scale is $k = 1 - 2r$.

5 Distance, penetration, and analytic derivatives

5.1 Separating axes and asymmetric intervals

For a unit axis u, write the projected intervals of two cores as $[a_-, a_+]$ and $[b_-, b_+]$. A gap exists if $a_+ \leq b_-$ or $b_+ \leq a_-$. When both directional depths are positive, the translation needed to separate along this axis is

$$d_u = \min(a_+ - b_-,\ b_+ - a_-).$$

This quantity is not generally the length of the interval intersection. For example, intervals $[-2, 2]$ and $[-1, 1]$ overlap in length two, but require translation three to separate. Computing the two directional depths also avoids assuming that support values of a tetrahedron in opposite directions are equal.

For three-dimensional convex polytopes, the candidate axes consist of face normals of either object and nonzero cross products of an edge direction of one with an edge direction of the other. These include all facet normals of the Minkowski difference. Eberly's separating-axis derivation. When the cores overlap, the minimum of d_u over these normalized axes gives their translational penetration depth: all Minkowski facets are tested, and an extra supporting direction cannot undercut the minimum boundary distance. The implemented kernel treats antiparallel axes as equivalent while retaining both interval directions.

For disjoint objects, the largest candidate-axis gap is a lower bound on Euclidean distance, not generally the distance itself. Replacing distance by this gap would distort rounded-shape penalties near edge and vertex contacts. The kernel therefore switches to feature enumeration whenever a separated pair remains close enough to contribute to the energy.

5.2 Feature enumeration and pruning

The closest points of separated convex polytopes can be obtained from vertex–face, face–vertex, and edge–edge candidates. Vertex projections are retained only when the orthogonal projection lies in the face polygon; segment distances include clamped endpoints and parallel cases. Degenerate closest-point sets can be represented by these feature combinations. The segment calculation follows the implementation pattern identified in Ericson, *Real-Time Collision Detection*, §5.1.9. Author's book site.

To prune candidates, suppose u separates K below L, with $h = \max_K u \cdot x$, $m = \min_L u \cdot y$, and gap $g = m - h > 0$. Let D be any current upper bound on distance. A minimizing pair (x, y) must satisfy

$$u \cdot x \geq h - (D - g), \qquad u \cdot y \leq m + (D - g).$$

Indeed $u \cdot x \geq u \cdot y - \|x - y\| \geq m - D$, and the second inequality follows symmetrically. Only features meeting these slabs need enumeration. After vertex–face checks reduce D, the slabs are rebuilt before checking edges. For edge midpoints m_1, m_2 and lengths ℓ_1, ℓ_2, their distance is at least $\|m_1 - m_2\| - (\ell_1 + \ell_2)/2$; edge pairs unable to beat D can therefore be skipped.

The code also uses bounding spheres and cached separating directions. A cached direction that still separates can initialize the same exact geometric distance problem without repeating the full axis scan. If its projected gap already exceeds the penalty cutoff $2r$, any lower bound suffices, because the pair energy is zero. These shortcuts affect workload rather than the intended objective.

The word "exact" in the search-kernel description means that these formulas compute the geometric distance or penetration rather than an approximation by a single convenient axis. Their JavaScript evaluation uses floating-point arithmetic, tolerances, and nearly parallel-axis thresholds. It is not an exact-arithmetic computation. The final proof is supplied by a different checker.

5.3 Translation and rotation gradients

A point p rigidly attached to a body with center c changes under a world-frame infinitesimal rotation ω by $\delta p = \omega \times (p - c)$. For a separated closest pair (p_a, p_b), set $n = (p_a - p_b)/\|p_a - p_b\|$.

The distance derivatives with respect to the centers are $n, -n$, and the rotation derivatives are $(p_a - c_a) \times n$ and $-(p_b - c_b) \times n$.

For penetrating cores, the active oriented support depth is $o = u \cdot (p_a - p_b)$. Its derivative contains both the changes in the support points and the rotation of the active axis. A face axis rotates with its parent body. For a normalized edge cross axis $u = \sigma w/\|w\|$, with $w = a \times b$,

$$\delta u = \frac{\sigma}{\|w\|}(I - uu^{\mathsf{T}})\delta w, \qquad \delta w = (\omega_a \times a) \times b + a \times (\omega_b \times b).$$

The analytic code includes these terms. Ignoring them would give incorrect orientation forces. At changes of the active support feature or active axis, the objective can be nonsmooth; a single smooth derivative need not exist. Finite-difference tests therefore distinguish smooth cases from feature-switching cases rather than interpreting every discrepancy at a kink as a calculus error.

6 Soft dynamics and numerical legalization

The documented objective has pair terms, wall terms, and inward pressure:

$$E = \sum_{i<j}(2r - \mathrm{sd}(K_i, K_j))_+^2 + \sum_{i,f}\left(h_{K_i}(N_f) + r - \frac{L}{2}\,\mathrm{off}_f\right)_+^2 + \mu L.$$

The wall expression is the support function of the rounded piece against a container face. Centers and orientations follow a heavy-ball update with momentum 0.85 and learning rate 0.04. Linear and angular deterministic velocities are capped at 0.03 per step. Container velocity has learning rate 0.02 and the same momentum and cap. Gaussian translational noise is added after the deterministic cap; angular noise has amplitude 1.5 times the translational amplitude. Consequently, the cap does not bound the full noisy displacement.

For transparency, the implemented container wall update is not precisely the derivative of the displayed energy: a wall violation q gives $\partial(q^2)/\partial L = -q\,\mathrm{off}_f$, whereas the code uses $-2q\,\mathrm{off}_f$. The piece derivatives match the stated penalties. This changes the relative pressure/wall balance in the container dynamics and must be preserved to reproduce the historical implementation. It does not affect the subsequent exact feasibility proof.

Fresh runs begin in a relatively spacious container with an approximate inscribed-ball packing fraction no greater than 0.07. Centers are rejection-sampled at least 0.7 internal units inside all container planes. Normalized four-dimensional Gaussian vectors initialize scalar-first quaternions, giving uniform random rotations. The simulator's seeded pseudorandom generator is `mulberry32`; Gaussian draws use a Box–Muller formula. The default annealed schedule is:

Phase	Steps	Rounding radius	Pressure	Translational noise
Ball compression	5,000	$r = 1/2$	$3\mu = 0.06$	$0.015(1-u)$
Sharpening	45,000	$(1-u)/2$	$\mu = 0.02$	$0.004(1-u)$
Rigid settling	3,000	0	$\mu(10^{-4})^u$	0

Here u is the elapsed fraction of a phase. Rotational velocities and rotational noise are disabled at the ball endpoint. Larger engine fresh runs, above population 16, use 70% of the default phase lengths. "Rigid" mode sets $r = 0$ throughout, but a systematic non-cube rigid-control comparison was planned and never executed.

Penalty dynamics can leave overlaps or protrusions. Both JavaScript and Python legalization therefore spread centers about their mean, keeping orientations fixed, until a floating SAT test reports no overlap at tolerance 10^{-12}. If $\bar{c}$ is the mean, the new center is $\bar{c} + \lambda(c_i - \bar{c})$, with $\lambda \geq 1$.

Scaling along each fixed pair-center ray is a natural correction because the Minkowski difference of the centered pieces is convex and contains the origin. Distinct centers eventually separate as λ grows; coincident centers are a degenerate input for which this mechanism cannot succeed.

After separation, the smallest container over common translations is fitted. If S_f is the maximum projection of every world vertex on N_f, the four-variable linear program is

$$\min_{t\in\mathbb{R}^3, z\in\mathbb{R}} z \quad \text{subject to} \quad S_f + N_f \cdot t \le z \,\mathrm{off}_f \,.$$

The internal container size is $L = 2z$. JavaScript enumerates four active container faces and solves the associated systems; Python uses HiGHS through SciPy. Both recompute z from the actual maximum support ratio at the chosen translation. This removes reliance on an underestimated solver objective, while the computations remain numerical. Python legalization accelerates the scale search using penetration/projection increments, with full SAT rechecks and a doubling/bisection fallback.

7 Hard-constraint local tightening

The file `src/exact.py` describes an "exact-contact" problem but implements floating-point constrained optimization. Its name should not be confused with the exact final certificate. The hard model minimizes L subject to containment of piece vertices and an explicit separating plane for every nearby pair. For convex objects, a separating plane is sufficient for nonoverlap; allowing its normal and offset to vary gives more flexibility than fixing one SAT axis.

Each round has $6n + 1 + 3m$ variables, where m is the number of included piece pairs: three coordinates and three rotation increments per piece, container size, and two normal parameters plus one offset per pair. Orientations are updated by $R_i = \exp([\phi_i]_\times)R_i^0$, with the reference rotations reset each outer round. A pair normal is parametrized in a tangent plane as

$$u(a,b) = \frac{u_0 + ae_1 + be_2}{\|u_0 + ae_1 + be_2\|}.$$

The pair constraints place every vertex of one piece on one side of this plane and every vertex of the other on the other side. Plane offsets are measured relative to the pair-center midpoint for conditioning. Containment constrains each world vertex against the container half-spaces. The analytic Jacobian includes the left Jacobian of $SO(3)$, Rodrigues rotation derivatives, and normalization derivatives of the plane normals.

A default trust-region half-width of 0.03 bounds center changes; rotation and plane-tangent parameters use three times that width. Container size lies between $L - 2\,\mathrm{trust}$ and $L + 0.01$. Pairs beyond a circumsphere-and-trust bound are omitted, and vertex inequalities too far from activity are pruned. Crucially, every proposed step is re-legalized and its container refitted before acceptance. Thus constraint screening and imperfect solver termination affect search quality, while the final record's validity remains independently checked.

Two local solvers are used. SLSQP solves the nonlinear constrained subproblem with an analytic Jacobian, up to 300 iterations and requested tolerance 10^{-15}, for up to 40 outer rounds. This requested tolerance is a stopping parameter, not a guarantee of 15 accurate decimal places. SciPy SLSQP documentation. Sequential linear programming instead solves one LP per round, using the current linearization $g(z_0) + J_0(z - z_0) \ge 0$ and the trust bounds. The LP is passed to HiGHS with a sparse constraint matrix. SciPy linear-programming documentation.

After either subproblem, acceptance depends on the legalized/refitted size, rather than the optimizer's nominal objective. SLP accepts an improvement exceeding 10^{-14}, grows the trust

width by a factor 1.5 up to its initial value, and halves it on rejection. SLSQP uses an acceptance threshold 10^{-13}, halves the width on rejection, and stops after three unsuccessful rounds or sufficiently small trust. These numerical thresholds are policies, not mathematical certificates of stationarity.

The batch pipeline screens up to six distinct raw candidates using SLP, then runs SLSQP on the two most promising by screened size. Distinctness there is only a size difference greater than 10^{-7}, so different geometries with the same size can be missed. The SLSQP polish in this legacy path restarts from the raw configuration, rather than necessarily continuing from the SLP endpoint. The autonomous engine uses SLP for gated candidates and SLSQP polishing at populations at most 14 when the SLP result is competitive. Its SLP time allowance is $30+4n$ seconds, with trust floor 10^{-6}; it can abort after round 15 if extrapolating the recent five-round gain for another 150 rounds would still miss the incumbent. This extrapolation is a speed heuristic and may discard a candidate that could improve after a later rearrangement.

8 Jiggling, structural change, and archive-based search

The engine keeps a best configuration per case and up to six alternative configurations. A hop expands the centers and container, softens the pieces, shakes at fixed container size, re-sharpens under pressure, settles the rigid pieces, legalizes, and tightens if competitive. Since $S_r \subseteq P$, softening an existing valid configuration immediately creates room without introducing larger pieces. Expansion creates additional slack. Shaking then explores how the relative arrangement can change.

The shaking criterion uses centers with their piece identities preserved. For all pairs neighboring in either the expanded reference or current arrangement, define

$$\Delta = \frac{1}{e_B}\left[\frac{1}{|\mathcal{N}|}\sum_{(i,j)\in\mathcal{N}}\Big(\|c'_i - c'_j\| - \|c_i - c_j\|\Big)^2\right]^{1/2}.$$

Neighboring here means center distance at most twice the piece circumradius. The reference is taken after initial expansion, so expansion alone does not count toward the requested rearrangement. The statistic is invariant under a common translation or rotation and is expressed in piece-edge units. It measures labeled relative center geometry; it does not detect an orientation change with fixed centers, prove a new contact graph, or certify escape from an optimization basin. Those distinctions matter when interpreting "jiggling" as a controlled search move.

Base hops draw Δ's target logarithmically uniformly from 0.04–0.6, expand by 1–5%, and start with rounding radius 0.06–0.20 in internal units. The last interval is 12–40% of the piece inradius, because that inradius is 0.5. Each initial shake block has 400 steps with translational noise 0.004. If the target is not reached, noise is multiplied by 1.5 and rounding radius by 1.3, capped at 0.4. After at most eight escalations, the algorithm proceeds even if the target remains unmet. The target is therefore a stopping objective subject to a finite budget, rather than a guaranteed property of every hop.

For populations at least eight, a focused hop is selected with probability 0.6: a random seed piece and its nearest centers form a cluster of between three and approximately $n/3$ pieces. The rest still receive 15% of the cluster's noise and all pieces still respond to contact forces; they are not fixed obstacles. Only neighbor pairs touching the cluster contribute to the focused change statistic. Re-sharpening uses $4000 + 150n$ steps, followed by $1000 + 25n$ rigid settling steps. The annealed morph noise starts at one quarter of the escalated shaking amplitude.

Other moves complement hops. Reinsert removes one to three pieces and adds replacements at candidate holes; ascend adds one piece to the best $(n-1)$-configuration; descend removes

one piece from the best $(n+1)$-configuration; fresh starts from random balls; tighten revisits a configuration inherited through monotonicity. Hole placement maximizes the minimum distance to existing centers among 4,000 sampled points that satisfy a wall margin. It is a center-based proposal heuristic, not an exact largest-polyhedral-hole calculation, and ignores orientation until subsequent dynamics. Structural addition/removal moves use one shake block rather than escalating until the ordinary hop target is met. Descend ranks every single-piece removal by translation-only container fit, then chooses uniformly from the three best removals before tightening.

The pool's fingerprint is the sorted list of all pairwise center distances. Configurations are treated as distinct when their sizes differ by more than 10^{-7}, or their fingerprints differ on average by more than 0.02 piece edges. This cheaply encourages diversity while ignoring piece permutations and common rigid motion. It is not a complete geometric invariant and does not define mathematical basins: different orientations and even different center arrangements can share such a fingerprint.

The scheduler fills missing cases first and then prioritizes configurations flagged for tightening. Otherwise, it favors the smallest score $(\text{effort}+30)(1+\text{since}/40)\xi/n$, where effort is accumulated worker elapsed seconds, since counts attempts without improvement, and ξ is uniform on 0.7–1.3. A configured focus share instead selects uniformly among designated benchmark cases. The initial ordinary-case move weights were hop 55%, reinsert 12%, fresh 13%, ascend 10%, and descend 10%. Following the one-hour experiment, production selected gentle hops and reduced fresh weight to 6%. At 17:20 on 11 October, the final ordinary-case weights became hop 76%, reinsert 4%, fresh 0%, ascend 10%, and descend 10%, with 25% focus on 12 selected benchmark cases. These weights are renormalized at the endpoint populations when ascend or descend is unavailable. Moreover, placeholder and missing-case rules can still invoke fresh starts despite their zero ordinary-case weight; the final policy did not disable them unconditionally.

Raw candidates are tightened only if their size lies within a case-specific relative gate of the incumbent, except descend and tighten moves, which are always tightened. The gate is 1.5 times the empirical 80th percentile of recent raw-to-tightened gains, clipped to 0.6–8%, with 3% used before sufficient history. It allocates work according to what tightening recently achieved, without assuming that raw candidate rank perfectly predicts final rank.

The archive enforces $U(n) \le U(n+1)$ after each accepted change. If violated, it deletes one piece from the higher-population best, refits the container, stores the result at the lower population, and queues tightening. The sweep runs downward, so an improvement can propagate through multiple populations. A separate record-stage check enforces monotonicity of certified sizes, where clearance adjustments could otherwise introduce tiny reversals. This ensures a logically coherent upper-bound table; it does not prove that any observed plateau belongs to the unknown optimal curve.

9 Constructive seeds and mixed-source lineage

Constructive arrangements provide reliable starting upper bounds when stochastic runs remain above a simple lattice. An edge-k tetrahedron has a standard subdivision into

$$\binom{k+2}{3} \text{ upright tetrahedra}, \qquad \binom{k}{3} \text{ inverted tetrahedra}, \qquad \binom{k+1}{3} \text{ octahedra},$$

all with unit edge. The volume identity is $k^3 = \binom{k+2}{3} + \binom{k}{3} + 4\binom{k+1}{3}$, because a unit octahedron has four times the tetrahedron's volume.

A unit tetrahedron fits in a unit octahedron: choose an octahedral face as its equilateral base and the centroid of the opposite face as apex. The perpendicular separation of these faces is

$2/\sqrt{6} = \sqrt{2/3}$, the height of a unit regular tetrahedron; the apex and base vertices lie in the octahedron, so their convex hull does too. Replacing each octahedral cavity by this tetrahedron gives

$$N_T(k) = \binom{k+2}{3} + \binom{k+1}{3} + \binom{k}{3},$$

namely 5, 15, and 34 pieces at $k = 2, 3, 4$. These are exact mathematical touching constructions; the generated machine records use numerical rotations and added clearance, which is a different representation of the upper bound.

For octahedra, write the unit piece as $\|x\|_1 \le a$, where $a = \sqrt{2}/2$. Place copies at am for integer triples satisfying $\|m\|_1 \le k-1$ and $\|m\|_1 \equiv k-1 \pmod 2$. Each lies inside $\|x\|_1 \le ka$. Distinct allowed centers are at L^1 distance at least $2a$, so interiors do not overlap. The count is $N_O(k) = (2k^3 + k)/3$, giving 6, 19, and 44 pieces. Cube grids provide k^3 pieces in edge k. The engine seeds these constructions at $k = 2, 3, 4$, trims oversized arrangements to population 40 by ranked removals, and uses the monotone archive rule to populate smaller cases.

These seeds constrain feasible upper bounds but need not be locally or globally optimal. The final 15-tetrahedron configuration is below edge three, and 16 octahedra fit below three although the 19-octahedron lattice fits at three. A structured construction can therefore be a useful starting point and a useful conjecture generator without fixing the best packing at every population.

Published coordinate files were an additional seed source. `scripts/literature.py` translates container conventions, rescales to internal units, normalizes quaternions, legalizes, and fits the smallest translated container. If a literature configuration is superior it becomes the incumbent; otherwise it enters the pool. Every derived move propagates its source lineage. This makes direct local refinements distinguishable from independently obtained configurations. Coordinate-free pictures were not reconstructed during the experiment, so several known numerical benchmark entries could not be used as seeds.

10 What certification proves

Search positions and quaternions are written as JSON numbers. Python first parses each numeric literal as a binary floating-point value; `Fraction` then treats that parsed value as an exact dyadic rational. The certificate consequently proves the configuration represented by those exact parsed values, not an intended decimal continuum value and not an ideal numerical touching limit.

For any nonzero rational quaternion $q = (w, x, y, z)$, the checker uses the homogeneous rotation formula

$$R(q) = \frac{1}{w^2 + x^2 + y^2 + z^2} \begin{pmatrix} w^2 + x^2 - y^2 - z^2 & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & w^2 - x^2 + y^2 - z^2 & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & w^2 - x^2 - y^2 + z^2 \end{pmatrix}.$$

This is exactly orthogonal with determinant one even when the stored quaternion is not exactly unit length. Hence each transformed vertex lies in the exact coordinate field and the piece is exactly congruent to the canonical unit-edge solid. The proof does not rely on floating normalization preserving rigidity.

Signs are decided algebraically. An element is written as $p + q\sqrt{5}$, with $p, q \in \mathbb{Q}(\sqrt{2})$. If p, q have opposing signs, compare p^2 with $5q^2$ in $\mathbb{Q}(\sqrt{2})$; signs in that smaller field are reduced to rational comparisons using a^2 and $2b^2$. Zero coefficients and equal-sign cases are handled directly. No floating tolerance enters the sign decision.

Proposition 3, per-record certificate. Let the file contain n nonzero quaternion poses and a positive stored size s_{full}. If every transformed piece vertex is strictly inside every exact container half-space, and for every pair the checker verifies a strictly separating plane, those pieces are pairwise disjoint unit-edge copies inside $s_{\text{full}}A$. Thus $s^*(A, B, n) \le s_{\text{full}}$.

Containment tests are $m_f \cdot (v - c_A) < s_{\text{full}}\beta_f$, with exact, unnormalized canonical face normals. Convexity extends these vertex inequalities to the entire piece. For separation, a floating SAT search proposes a direction; its components are rationalized with denominator limit 10^{12}. Exact vertex projections are then computed and an exact midpoint offset is proposed. Every vertex is checked with exact signs on its required side. A mistaken floating proposal can make certification fail, but cannot produce a false success after these exact checks.

The plane normal is rational; its midpoint offset generally lies in $\mathbb{Q}(\sqrt{2}, \sqrt{5})$. Thus "rational separating plane" in project summaries is more precisely a plane with rational normal and exact algebraic offset. Floating comparisons also select the candidate projection extremes, but every projection is subsequently checked, so a mistaken choice cannot bypass the proof. The implementation regenerates its witnesses during checking and prints a summary rather than serializing every plane in a certificate file.

Record preparation spreads centers to target pair gaps and adds wall clearance, both nominally 10^{-7} in unit-edge Euclidean length. The final container size is rounded upward at the thirteenth decimal. The tetrahedral wall cost alone is $g/\rho_T \approx 4.899g$ in edge ratio, explaining why the initial 10^{-6} convention was reduced. Pair clearance uses SAT projection gaps, which are lower bounds on Euclidean separation, and omits pairs beyond twice the circumradius. Such far pairs are separated, but a pair just beyond that cutoff is not necessarily covered by the claimed global gap target. The exact checker guarantees strict separation of every pair, independently of that target.

The numeric minimum margins printed by the checker are approximate diagnostics; their signs are proven by the exact inequalities. The recorded `s_tight` is a numerical optimizer estimate, `s_full` is the certified upper bound, and a fitted `closed_form_conjecture` remains conjectural. The displayed five-decimal "+" values are truncations of `s_full`; removing the plus sign would incorrectly present a smaller, uncertified bound. The certificate counts the actual `pieces` list; a dataset audit must additionally compare its length with the declared population field.

The field used for canonical coordinates and rational rotations does not constrain the algebraic form of an unknown optimum. An exact-contact optimizer could require rotations and edge ratios in a larger extension, or values with no simple radical expression. Matching a small-coefficient formula numerically is useful for generating contact-equation conjectures, not for proving that formula.

11 Validation and experimental comparisons

For this manuscript package, a fresh independent replay of the frozen record set ran both checkers and verified that every declared population equals the actual pose-list length. All 351 records passed the exact checker, floating checker, and schema count check, with zero failures, in 351.38 elapsed seconds. The accompanying verification log records per-file SHA-256 values and checker outputs. The original geometry test was also rerun successfully: 731 separated cases, 769 penetrating cases, and all 1,500 gradient cases passed, with maximum smooth gradient discrepancy approximately 1.43×10^{-9}. These are new verification runs of archived inputs, not a repetition of the historical 121,109-attempt search.

The recorded geometry tests sampled 1,500 pairs across all 25 ordered solid pairs: 731 separated and 769 penetrating pairs. Separated closest points were checked against supporting-plane/KKT conditions; penetration was checked by translation and comparison with 12,000 sampled direc-

tions per tested pair. Smooth analytic gradient errors reached approximately 1.43×10^{-9} at worst in the recorded run. Random-direction penetration tests are empirical checks, while the finite-axis/Minkowski derivation is the underlying mathematical argument. The KKT checks use numerical tolerance and should not be confused with the exact record certificates.

Tightening Jacobians were checked by central differences for every solid. Exact field arithmetic was compared with 80-digit decimal calculations on 3,000 random elements, including near cancellations. Two independent external configurations passed the exact checker: Nakajima's 12-cube file and the Hyra-results 21-octahedron-in-cube file at edge 2.592849935818519. Deliberately damaged variants, including shrunken containers, pushed neighbors, coincident pieces, and pieces outside a wall, were rejected. These tests address representation, normalization, and failure detection; they do not prove the optimizer's superiority over all competing methods.

In a direct cube comparison, the original and generalized simulators both reached edge two for population eight at seeds one and two. At population nine their raw endpoints differed: 2.828428 versus 2.742095 at seed one, and 2.754057 versus 2.828427 at seed two. Thus "same behavior" means comparable outcomes and recovered benchmark constructions, not identical seed-by-seed trajectories. The measured runtimes improved by factors approximately 1.3–2.1 in those four comparisons.

Profiling identified feature enumeration and legalization as useful optimization targets. Rebuilding the slabs after vertex–face checks, midpoint edge rejection, and allocation-free segment distance reduced one 12-dodecahedron test from 2,669 to 1,696 ms with the same endpoint. Newton-style legalization reduced a 20-dodecahedron tightening example from an unfinished 120-second run to a completed 24.5-second run. These are particular comparisons, not universal speed factors.

For 108 matched raw candidates tightened by both solvers, 87 final sizes agreed, SLSQP was smaller in 13, and SLP in eight. Median elapsed tightening time was 18.2 seconds for SLSQP and 12.0 for SLP; SLP was nevertheless slower in 45 pairs. The largest reported SLSQP advantage was approximately 1.2% in size, with substantially greater runtime. These results motivate selective polishing and reject the stronger claim that one solver dominates the other. Raw rank also reversed after tightening: one 13-dodecahedron candidate at roughly 3.0001 tightened to 2.968 and beat another starting at 2.987 that tightened to 2.980.

The one-hour hop-setting experiment randomized the variant independently at each hop. The observed elapsed-time-normalized outcomes were:

Variant	Hops	New archive bests	Summed worker hours	Bests per worker hour	Reported 90% Poisson interval
Base	196	4	0.169	23.7	8.1–54.2
Jiggle: noise ×3, shake block ×1.5	185	5	0.186	26.8	10.6–56.4
Expand: initial expansion 6–15%	184	5	0.263	19.0	7.5–40.0
Volume: three raw hops per attempt	194	14	0.680	20.6	12.4–32.2
Gentle: target 0.02–0.10, expansion 0.5–3%	190	18	0.384	46.9	30.3–69.6

Gentle also starts from the best with probability 0.9, compared with 0.6 in base. It changes several settings together, so the experiment cannot isolate the effect of target size alone. Randomized interleaving reduces time and case-allocation confounding, but all arms improve a shared

evolving archive and observations need not be independent. The intervals overlap; the experiment supports choosing the highest observed yield for production, rather than a definitive significance claim or universal optimum. Counting bests exceeding 10^{-4} gave similar rates for jiggle and gentle, illustrating dependence on the improvement metric.

Whole-run cloud attempts gave 33,457 hops with 2,157 improvements, 7,698 ascends with 512, 6,775 reinserts with 403, 6,626 fresh runs with 84, 5,619 descends with 99, and 251 flagged tightenings with 151. Ascend was productive while filling the archive; flagged tightenings are selected loose configurations and cannot be fairly compared as if they were random moves. During the final 14-hour window, hops produced more improvement events per worker hour than ascend, while ascend produced roughly three times the total logarithmic shrink per worker hour. Fresh runs produced no improvements during 2.2 worker hours in that window. These outcomes concern this evolving archive and its scheduler; they do not prove that restarts can be omitted in unrelated packing problems.

12 Final records, negative results, and limitations

The final archive contains 351 record files with exact and floating checker successes recorded for every case. Selected full certified edge ratios are listed below; numerical touching estimates are supplied only where useful for interpreting the construction. The authoritative decimals are the stored values and their exact dyadic interpretation in the checker.

Problem	Population	Certified s_{full}	Numerical s_{tight}	Interpretation
Tetrahedra in tetrahedron	2	1.9535648569341	1.953564244562	Two pieces below edge two
Octahedra in octahedron	2	1.9331199262199	1.933119581160	Two pieces below edge two
Icosahedra in icosahedron	2	1.9630862485264	1.963086047134	Two pieces below edge two
Dodecahedra in dodecahedron	2	1.9875982173898	1.987598085505	Two pieces below edge two
Tetrahedra in tetrahedron	8	2.3874266594137	2.387425886723	Numerical match to $(4+\sqrt{10})/3$
Tetrahedra in tetrahedron	15	2.9888335999753	2.988832622915	Improves the edge-three subdivision seed
Octahedra in octahedron	10	2.5000004286608	2.500000000000	End of the observed 7–10 plateau
Octahedra in octahedron	16	2.9946516508438	2.994651162686	Below the neighboring edge-three plateau
Icosahedra in icosahedron	12	2.6180342210669	2.618033988750	Numerical match to φ^2
Dodecahedra in dodecahedron	12	2.7639321915113	2.763932022500	Numerical match to $5-\sqrt{5}$
Cubes in cube	11	2.8817115130006	2.881711124829	Refinement with literature lineage
Cubes in cube	12	2.9315149711166	2.931514577965	Reproduction of the same touching-scale benchmark
Octahedra in cube	21	2.589573377004	2.589572965578	Refinement with literature lineage
Octahedra in cube	24	2.6279229560838	2.627922547582	Literature-seeded local refinement
Octahedra in cube	29	2.8284275556868	2.828427124746	Numerical match to $2\sqrt{2}$

The complete numerical table is the frozen summary CSV. The lower `s_full` of the present 12-cube file relative to Nakajima's released file is largely explained by the reduced clearance convention; it should not be advertised as a new geometric improvement over the same touching configuration. Other literature comparisons require matching edge conventions, clearance, truncation, source dates, and full-precision coordinates. The methods contribution remains useful even where another researcher has subsequently obtained smaller bounds.

The autonomous phase used 60,426 logged cloud attempts, 14,148 laptop attempts, and 46,535 Actions attempts, totaling 121,109. Cloud search recorded approximately 102.1 summed worker elapsed hours and laptop search approximately 23.6; the 14 Actions runs scheduled about 93 worker hours. These are not interchangeable with measured processor utilization or machine wall time. The archive records 3,406 local improvements and 661 imported improvements, totaling 4,067 accepted events; a remote merge can summarize several remote improvements. Logging and initial coverage also affect what constitutes an improvement event.

Negative results are part of the reproducible method. Harder shaking, larger expansions, and three raw candidates per attempt did not improve the measured best-per-time yield over gentle perturbations in the short experiment. Large hop targets were less productive in observational logs, but the changing archive and policy confound those comparisons. Pool starts and global hops gave lower observed yields than best starts and focused hops. Neither SLSQP everywhere nor SLP everywhere was consistently best. Direct SLSQP on every possible deletion was too expensive, and screened deletion was adopted. Exact face enumeration without a numerical prescreen made checker startup a bottleneck; deriving and verifying the faces with a prescreen fixed this without weakening the exact final inequalities.

Several planned experiments were not run: the systematic rigid-control ablation, a full 25-pair sweep, larger fresh-run budgets, a complete initial descend chain for every problem, and exact-contact reconstruction of guessed formulas. No interval global optimization or optimality proof was attempted. Consequently, the method's soft phase has a principled geometric motivation and successful outcomes, but its causal advantage for non-cube solids has not been measured against a budget-matched rigid control.

Operational failures included two cloud suspensions, a reboot, a closed-form finder crash on a degree-one polynomial, a race between publishers repaired by a lock, killed laptop background workers replaced by bounded runs, unavailable bundle-transfer storage replaced by GitHub synchronization, and unreliable Actions cron triggering replaced by manual dispatch logic. Resume-from-disk archives and frequent checkpoints limited lost work. These incidents affect experiment duration and scheduling; they do not invalidate the stored configurations. The source still contains some historical simplifications: the record `method` string mentions SLSQP even for predominantly SLP-tightened records, and the publisher can write a failed candidate with a failure marker rather than automatically repairing it. Certification status must therefore be checked, rather than inferred from a record's existence.

The most important scientific limitations are distinct. A feasible placement is not a proven global optimum. A converged local solver is not necessarily a certified stationary point. A repeated closed-form match is not an exact-contact derivation. Plateaus of upper bounds can be caused by deletion from one construction, and cannot by themselves prove plateaus of s^*. Center-distance shaking and pool fingerprints do not measure full orientational order. Finite walls and populations through 40 do not establish an infinite packing density or thermodynamic phase. Artificial damping, penalty pressure, rounding, and annealed noise do not constitute physical crystallization dynamics. These configurations can motivate studies of shape-dependent order and confinement, but grain formation, material growth laws, and crystallization mechanisms require additional physical models and evidence.

13 Reproduction and inspectable outputs

Reproducing a certificate is simpler than reproducing the full adaptive search. Both checkers require only the Python standard library and the repository. A reader can clone the project, select the immutable source commit, and check one file:

```
git clone https://github.com/alejandrozu/platonic-packing.git
cd platonic-packing
git checkout e15b496ae7306287e0e70bb5b14be79f5dfdde84
python3 solids.py
python3 certify_exact.py records/tetintet/tetintet_n08.json
python3 verify.py records/tetintet/tetintet_n08.json
```

The exact self-check rebuilds canonical geometry. The certificate proves the stored placement; the floating verifier supplies a separately implemented tolerance-based check. To inspect an external fixture, the test directory contains the upstream record files and `test/run_all.sh` orchestrates geometry, Jacobian, arithmetic, external-record, and deliberately damaged-record tests. The suite's exact file coverage should be read from its code: passing a suite does not automatically imply that every later added record was included.

Search requires Node.js at least 18 and Python at least 3.10, with NumPy and SciPy; figures use matplotlib and formula hints use mpmath. The autonomous engine depends on Unix `fc ntl` locking, and the shell supervisor assumes a Unix-like environment. Native Windows can execute the standard-library checkers, while autonomous reproduction should use Linux or WSL. The following batch commands are executable from the selected checkout and write local search outputs:

```
python3 -m pip install numpy scipy matplotlib mpmath
node scripts/batch.js tetrahedron same 7 1 8 1 improved
python3 scripts/pipeline.py tighten 2 tetrahedron
python3 scripts/pipeline.py descend tetrahedron
```

The two seed arguments are an inclusive range, so the example runs seeds one through eight. The batch runner caches existing seeds. The tightening and descend commands can also read earlier cached runs in the checkout; a genuinely isolated experiment should begin from a clean copy of the required code and a deliberately selected dataset rather than assuming the cache is empty.

For a bounded autonomous experiment in Linux or WSL, a separate archive prevents modification of the historical archive:

```
mkdir -p work/reproduction-state
export PP_STATE="$PWD/work/reproduction-state"
cp state/policy.json "$PP_STATE/policy.json"
python3 scripts/engine.py seed
export PP_DEADLINE="$(python3 -c 'import time; print(time.time()+600)')"
python3 scripts/engine.py work 0
python3 scripts/engine.py check
python3 scripts/engine.py status
```

This performs a local ten-minute worker run after seeding; it does not invoke the publishing supervisor. The ordinary `run_forever.sh` also launches publishing that can commit and push, so it is inappropriate for a read-only reproduction of the frozen archive. The bounded example's scheduler and noise seeds depend on process identity and time; future runs are not expected to reproduce the exact historical order of attempts. Individual simulator requests carry explicit seeds, but archive state, alternative pools, scheduling, concurrent merges, and numerical-library

versions affect the overall trajectory. Alternative pools were not tracked in Git, and the repository does not pin the historical dependency environment. These are material limits to bitwise search reproduction, while certificate reproduction remains tied to the immutable coordinate files and checker.

The repository includes raw run files, tightened batch results, best archive entries, an attempt log, per-case coordinate JSON, checker outputs, PNG pictures, and an interactive three-dimensional viewer. The viewer allows rotation, slicing, piece selection, neighbor inspection, and comparison of size and increment curves. It supports interpretation but is not itself the certificate: a convincing image cannot rule out microscopic overlaps. Each scientific claim of feasibility should point to the coordinate file, source commit, and checker result. Exact witnesses, full contact equations for the conjectured sizes, pinned runtime environments, and a budget-matched rigid-control study are the next methodological additions that would most strengthen this evidence base.

References and source map

1. Zarzuelo, A. *Platonic packing*, frozen repository `e15b496ae7306287e0e70bb5b14be79f5dfdde84`, 2026. Immutable repository. Primary project source; methods and results are directly reconstructable from this snapshot.

2. Project narrative and corrections: docs/FINAL_REPORT.md, especially Parts I and II. Historical claims in Sections 2 and 11–12 of this manuscript are attributed to this record of the original experiment.

3. Search geometry and dynamics: src/geom.js, src/sim.js. Hard tightening: src/exact.py. Record preparation: src/claim.py.

4. Exact canonical solids: solids.py. Exact checker: certify_exact.py. Floating checker: verify.py.

5. Autonomous search and policy: scripts/engine.py, state/policy.json. Constructive seeds: scripts/constructions.py. Legacy workflow: scripts/pipeline.py.

6. Nakajima, Y. *Twelve unit cubes in a cube of side 2.9315: soft-to-rigid packing*, v1.0.0, 2026. Software and primary description. The upstream repository identifies the release DOI as 10.5281/zenodo.23248095; bibliographic metadata here follows that upstream attribution. Method and MIT-licensed code credit retained.

7. Wales, D. J., and Doye, J. P. K. "Global Optimization by Basin-Hopping and the Lowest Energy Structures of Lennard-Jones Clusters Containing up to 110 Atoms." *Journal of Physical Chemistry A* **101** (1997), 5111–5116. DOI: 10.1021/jp970984n.

8. Eberly, D. *Intersection of Convex Objects: The Method of Separating Axes.* Geometric Tools, document revised 20 June 2022. Author's derivation.

9. Ericson, C. *Real-Time Collision Detection.* Morgan Kaufmann, 2005, ğ5.1.9. Author's book page.

10. SciPy community. `minimize(method='SLSQP')` and `linprog`, official documentation, accessed 11 October 2026. These links document solver interfaces; they do not pin the runtime versions used in the historical experiment.

11. Egeblad, J., Nielsen, B. K., and Brazil, M. "Translational packing of arbitrary polytopes." *Computational Geometry* **42**(4) (2009), 269–288. DOI: 10.1016/j.comgeo.2008.06.003.

12. Romanova, T., Bennell, J., Stoyan, Y., and Pankratov, A. “Packing of concave polyhedra with continuous rotations using nonlinear optimisation.” *European Journal of Operational Research* **268**(1) (2018), 37–53. DOI: 10.1016/j.ejor.2018.01.025.

13. Callisaya, H. F., and Kari, E. R. V. “Packing Polyhedra within Convex Sets.” *Trends in Computational and Applied Mathematics* **26**, e01823 (issue year 2025; online collection publication 19 January 2026). DOI: 10.5540/tcam.2025.026.e01823.